

Lesson 12: Introduction to Classes

C++ is a bunch of small additions to C, and one major addition. This one addition is the object-oriented approach. As its name suggests, this deals with objects. Of course, these are not real-life objects. Instead, these objects are the essential definitions of real world objects. Classes are collections of data related to a single object type. Classes not only include information regarding the real world object, but also functions to access the data, and classes possess the ability to inherit from other classes. Inheritance is covered in a later lesson.

What is this mysterious beast? Well, it is not only a collection of variables under one heading, but it is a collection of functions under that same heading. If a class is a house, then the functions will be the doors and the variables will be the items inside the house. They usually will be the only way to modify the variables in this structure, and they are usually the only to access the variables in this structure. The idea to make programs more modular - encapsulation. A section of code will have its own functions and variables that control what it can do, and it does not require anything outside of itself to function. While the class may require initialization with data, it does not require outside variables or functions to manipulate the data. That allows programs to reuse the same code more easily.

The syntax for these classes is simple. First, you put the keyword 'class' then the name of the class. Our example will use the name computer. Then you put an open bracket. Before putting down the different variables, it is necessary to put the degree of restriction on the variable. There are three levels of restriction. The first is public, the second protected, and the third private. For now, all you need to know is that the public restriction allows any part of the program, including that which is not part of the class, access the variables specified as public. The protected restriction prevents functions outside the class to access the variable. The syntax for that is merely the restriction keyword (public, private, protected) and then a colon. Finally, you put the different variables and functions (You usually will only put the function prototype[s]) you want to be part of the class. Then you put a closing bracket and semicolon. Keep in mind that you still must end the function prototype(s) with a semi-colon.

Classes should always contain two functions: the constructor and destructor. The syntax for them is simple, the class name denotes a constructor, a ~ before the class name is a destructor. The basic idea is to have the constructor initialize variables, and to have the destructor clean up after the class, which includes freeing any memory allocated. The only time the constructor is called is when the programmer declares an instance of the class, which will automatically call the constructor. The only time the destructor is called is when the instance of the class is no longer needed. When the program ends, or when its memory is deallocated (if you do not understand the deallocation part, do not worry). Keeps in mind this: NEITHER constructors NOR destructors RETURN AN ARGUMENT! This means you do not want to try to return a value in them.

The syntax for defining a function that is a member of a class outside of the actual class definition is to put the return type, then put the class name, two colons, and then the function name. This tells the compiler that the function is a member of that class.

For example:

```
#include <iostream>

using namespace std;

class Computer // Standard way of defining the class
{
```

```

public:
    // This means that all of the functions below this(and any variables)
    // are accessible to the rest of the program.
    // NOTE: That is a colon, NOT a semicolon...
    Computer();
    // Constructor
    ~Computer();
    // Destructor
    void setspeed ( int p );
    int readspeed();
protected:
    // This means that all the variables under this, until a new type of
    // restriction is placed, will only be accessible to other functions in the
    // class. NOTE: That is a colon, NOT a semicolon...
    int processorspeed;
};
// Do Not forget the trailing semi-colon

Computer::Computer()
{
    //Constructors can accept arguments, but this one does not
    processorspeed = 0;
}

Computer::~~Computer()
{
    //Destructors do not accept arguments
}

void Computer::setspeed ( int p )
{
    // To define a function outside put the name of the class
    // after the return type and then two colons, and then the name
    // of the function.
    processorspeed = p;
}
int Computer::readspeed()
{
    // The two colons simply tell the compiler that the function is part
    // of the clas
    return processorspeed;
}

int main()
{
    Computer compute;
    // To create an 'instance' of the class, simply treat it like you would
    // a structure. (An instance is simply when you create an actual object
    // from the class, as opposed to having the definition of the class)
    compute.setspeed ( 100 );
    // To call functions in the class, you put the name of the instance,
    // a period, and then the function name.
    cout<< compute.readspeed();
    // See above note.
}

```

This introduction is far from exhaustive and recommends practices that are not always the best option. This was done for simplicity. For more detail, I suggest asking questions on our forums and getting a book recommended by our book reviews.